



TAREA 3 CONCEPTOS DE UNIDAD 2

jun/07/2026



cesun
Universidad

CARLOS LEONEL

DESARROLLO DE SOFTWARE BACKEND

Bailon Yira

DESCRIPCION

Las estructuras de datos lineales son formas organizadas de almacenar y manipular datos en memoria. Su comportamiento depende de como se ingresan y se eliminan datos.

Estructuras	Descripción	
Listas	Secuencias de elementos similares a los arreglos, pero con mayor flexibilidad para crecer o encogerse dinámicamente. En muchos lenguajes modernos, los términos "Arreglo" y "Lista" se usan casi como sinónimos.	frutas = ["Manzana", "Banana", "Naranja"] puts frutas[1] # Salida: Banana
Arreglos	Colecciones ordenadas de elementos que generalmente ocupan espacios contiguos en memoria. Cada elemento tiene un índice numérico (empezando habitualmente en 0). Su tamaño puede ser fijo o dinámico dependiendo del lenguaje.	frutas = ["Manzana", "Banana", "Naranja"] puts frutas[1] # Salida: Banana
Pilas	Estructuras que siguen el principio LIFO (<i>Last In, First Out</i> - Último en entrar, primero en salir)	pila_libros = [] pila_libros.push("Libro 1") # Agregar al final pila_libros.push("Libro 2")
colas	Estructuras que siguen el principio FIFO (<i>First In, First Out</i> - Primero en entrar, primero en salir).	fila_banco = [] fila_banco.push("Cliente A") # Entra a la fila fila_banco.push("Cliente B")
Hashes	es una colección indexada donde, a diferencia de los arreglos que usan números (0, 1, 2...), se utilizan claves personalizadas (cadenas de texto, símbolos o números) para acceder a un valor.	# 1. Creación de un Hash usuario = { "nombre" => "Ana", "edad" => 28, "rol" => "Developer" } # 2. Lectura (Acceso a un valor mediante su clave) puts usuario["nombre"] # Salida: Ana
Hashes anidados	Los hashes anidados ocurren cuando el valor asociado a una clave es, a su vez, otro hash u otra colección de datos. Es la base de estructuras complejas	# Estructura de una empresa con hashes anidados empresa = { "departamento_it" => { "jefe" => "Carlos Romero", "empleados" => 5 }, "departamento_ventas"

	como el formato JSON , ampliamente usado en desarrollo web.	=> { "jefe" => "Laura Gómez", "empleados" => 12 }
Iteración de colecciones de datos	Iterar significa recorrer cada uno de los elementos de una colección (como un arreglo o un hash) para procesarlos uno por uno.	# Iterando un Arreglo precios = [100, 250, 500] precios.each do precio puts "El precio con IVA es: #{precio * 1.16}" end # Iterando un Hash capitales = { "México" => "CDMX", "España" => "Madrid", "Colombia" => "Bogotá" } capitales.each do pais, capital puts "La capital de #{pais} es #{capital}." end
Controles de flujo	Los controles de flujo permiten romper la ejecución lineal de un programa, tomando decisiones o repitiendo bloques de código según se cumplan o no ciertas condiciones.	
If/else	Evalúa una condición primaria; si no se cumple, pasa al bloque alternativo.	calificacion = 85 if calificacion >= 90 puts "Excelente" elsif calificacion >= 70 puts "Aprobado" else puts "Reprobado" end
Switch/Case	deal para evaluar una sola variable frente a múltiples opciones posibles de forma más limpia que usar muchos if encadenados.	día_semana = "Sábado" case día_semana when "Lunes", "Martes", "Miércoles", "Jueves", "Viernes" puts "A trabajar" when "Sábado", "Domingo" puts "Fin de semana, a descansar" else puts "Día inválido" end
do	Asegura que el bloque de código se ejecute al menos una vez antes de evaluar la condición.	loop do puts "Este mensaje se lee al menos una vez." condicion = false break if !condicion # Salimos del bucle si la condición ya no se cumple end
while	Ejecuta el bloque de código mientras la condición sea verdadera .	contador = 1 while contador <= 3 puts "Contando con while: #{contador}" contador += 1 end
Until	Ejecuta el bloque de código hasta que la condición se vuelva verdadera (es decir, se ejecuta mientras sea <i>falsa</i>).	energia = 3 until energia == 0 puts "Cargando datos... Energía restante: #{energia}" energia -= 1 end